

ASHAKids | Auditoría de aceptación del núcleo y reproducción independiente

AUDIT-2026-10-09-08 — verificación del núcleo, PDF, mensajes y análisis de incidente en entorno independiente

Fecha: 2026-10-09. Responsable: reproducción asistida en el entorno independiente de HailQueso. Entrega académica: 18:00 Lima.

Repositorio: <https://github.com/sromansilva/ashakids-platform> . Rama: dev. Aplicación reproducida: dev/b3f84bd828c2002c7f80466410251d9df155550c (V13_Integracion_Aceptacion_Nucleo / PR #107 integrado).

Árbol de trabajo: clon independiente con dependencias instaladas en backend y frontend. Verificación ejecutada en hardware, sistema operativo, cluster y puertos independientes al entorno histórico de Piero. Sin datos reales ni interacción con Supabase compartido.

01. Dictamen ejecutivo

Se reprodujo exitosamente el recorrido completo de aceptación del núcleo clínico, exportación de reportes PDF y mensajería autenticada en un entorno de desarrollo independiente (ordenador de HailQueso con PostgreSQL 18.4 en puerto 5433). Se utilizó una base de datos local nueva para aceptación (ashakids_test_accept07_hq) y otra separada para la suite completa de regresión (ashakids_test_regress08).

La suite de 92 respuestas HTTP del guion de entrega se ejecutó sin fallos. Las pruebas automatizadas unitarias y de integración backend alcanzaron 119 casos aprobados, 25 omitidos y 19 advertencias. El frontend superó 232 pruebas de componentes, 25 de enrutamiento, chequeo estático y compilación de producción. Se identificó la limitación de ejecución en navegador debida a la indisponibilidad de descarga del driver del gestor Playwright (error 404 del CDN azureedge), por lo que la fase 5 se mantiene honestamente abierta en el plano de interacción visual real.

Indicador	Resultado verificado	Alcance / Entorno
Backend regresión	119 aprobadas, 25 omitidas, 19 advertencias	PostgreSQL 18.4 local (puerto 5433), BD regress08
Frontend componentes	232 aprobadas, 0 fallos (12 suites)	jsdom con mocks HTTP, tipos y check conformes
Frontend enrutamiento	25 aprobadas, 0 fallos (5 suites)	Node experimental strip-types sobre routing.test.mjs
Recorrido entrega HTTP	92 respuestas verificadas (0 fallos)	API loopback 8001 / PG 18.4 local 5433 (accept07_hq)
Integridad PDF	Estructura %PDF-, 4 campos cotejados	Generado en memoria por ReportLab 4.5.1, sin notas privadas
Interfaz en navegador	Bloqueada por driver Playwright (404 CDN)	Documentada sin evasión de origen 5174 ni puertos

02. Arquitectura, estructura y reproducción independiente

Se mantiene la arquitectura oficial: React 18 + Vite -> API HTTP / cookies HttpOnly -> FastAPI -> servicios / capas de dominio -> SQLAlchemy 2.x / asyncpg -> PostgreSQL local.

La reproducción en este equipo (HailQueso) comprobó que el proyecto no contiene dependencias ocultas del entorno de Piero (puerto PostgreSQL 6544 o usuario Piero):

- Entorno de ejecución: Node v22.15.0, npm 10.9.2, Python 3.13.7 en backend/.venv.
- Dependencias backend instaladas: FastAPI 0.143.0, SQLAlchemy 2.0.44, asyncpg 0.31.0, ReportLab 4.5.1, tzdata 2026.5, pytest 9.1.1, pypdf 6.20.0.
- Servidor de base de datos: PostgreSQL 18.4 local escuchando en el puerto 5433.
- Rol de ejecución: ashakids_audit_runtime con contraseña sintética, atributo BYPASSRLS, NOSUPERUSER, NOCREATEDB, NOCREATEROLE. Propietario administrativo postgres utilizado exclusivamente para creación de bases y esquemas.
- Esquema relacional: 26 tablas públicas, 178 columnas, 221 restricciones (claves primarias, foráneas, chequeos, nulabilidad) y 71 índices restaurados desde shared-public-schema.sql (versión portable idempotente).

Componente físico	Métrica verificada	Evidencia
Tablas en esquema public	26 tablas verificadas	schema-local.json
Columnas totales	178 columnas	schema-local.json
Restricciones activas	221 restricciones	schema-local.json
Índices relacionales	71 índices	schema-local.json
Modelos de dominio	19 modelos ORM coincidentes al 100%	Inspección AST y ORM

03. APIs funcionando: identidad y pacientes

Se verificó el flujo completo de autenticación y autorización por roles (ADMIN, PADRE, TERAPEUTA) sobre la base local ashakids_test_accept07_hq:

Método y ruta	Estado	Resultado
POST /api/v1/auth/login	200 / 401	Sesiones emitidas para identidades válidas; rechazo ante clave errónea
GET /api/v1/usuarios	200 / 403	ADMIN lista cuentas; PADRE y TERAPEUTA reciben 403 Forbidden
POST /api/v1/usuarios	201	ADMIN crea cuenta sintética de tutor (p90003)
PATCH /api/v1/usuarios/{id}	200	Suspensión lógica de cuenta y reactivación administrativa
GET /api/v1/auth/me	401	Sesión invalidada de inmediato tras suspensión de cuenta
POST /api/v1/pacientes	201	Tutor autorizado da de alta expediente de paciente propio
POST /api/v1/tratamientos	201 / 403	ADMIN asigna terapeuta al paciente; familia recibe 403 al intentarlo
GET /api/v1/pacientes/{id}	404	Tutores y terapeutas no asignados no pueden consultar el expediente

Antes de cualquier mutación clínica en la base, el guion ejecutó confirm_api_database, comprobando mediante consulta directa a PostgreSQL que la cookie emitida por la API existiera en sesiones_autenticacion, estuviera activa y no vencida, impidiendo cualquier ejecución accidental contra una base ajena.

04. APIs funcionando: citas, reportes y mensajes

El recorrido integró la reserva de citas, gestión de sesiones, emisión de reportes clínicos en PDF y mensajería privada bidireccional:

Recorrido / Capacidad	Respuesta	Comportamiento verificado
Reserva de cita	201 / 409	PADRE reserva en tratamiento autorizado; conflicto horario rechazado con 409
Confirmación de cita	200 / 403	Terapeuta asignado confirma (200); tutor no puede auto-confirmar (403)
Inicio y ciclo de sesión	201 / 200	Sesión creada por terapeuta; bloqueo preventivo antes de hora (409); inicio validado
Reporte clínico	200 / 403	Terapeuta guarda los 4 campos estructurados; tutor no puede editar reporte (403)
Cierre con asistencia	200	Cierre exitoso con ASISTIO; estado de sesión FINALIZADA y reserva COMPLETADA
Descarga PDF	200 / 404	Tres roles autorizados obtienen binario %PDF-; usuarios no asignados reciben 404
Chat entre participantes	200 / 201	Contactos asignados visibles; apertura concurrente idempotente; intercambio de mensajes
Paginación y cursores	200	Paginación mediante cursor before_id sin solapamiento ni repeticiones
Persistencia tras relogin	200	Logout y nuevo login conservan idéntico historial de mensajes y contenido de reporte
Restricciones de estado	403 / 409	Tratamiento finalizado bloquea envío (403); chat archivado bloquea envío (409); lectura preservada

El PDF exportado por la API (reporte-sesion-sintetica.pdf) fue analizado mediante pypdf: consta de 1 página, contiene el texto íntegro de los cuatro campos guardados (observaciones iniciales con código de auditoría, objetivos trabajados, nivel de ayuda, próximos pasos), cabeceras Cache-Control: no-store y X-Content-Type-Options: nosniff.

05. Pendientes, capacidades no certificadas e interfaz real

Capacidad	Estado actual	Acción / Justificación
Interfaz en navegador	Bloqueada en gestor	browser_subagent reportó fallo de instalación del driver Playwright (404 CDN); no se evadieron puertos
Despliegue en hosting	Pendiente	Supeditado a estabilidad comprobada y selección final del usuario (F7-02/03)
Persistencia Mundo ASHA	En etapa propia	Los juegos interactivos se desarrollan en la etapa específica MA-01..MA-07
Módulo de pagos	Excluido	Retirado formalmente de la implementación por indicación académica
Extensiones avanzadas	Por confirmar	Teleconsulta WebRTC, reconocimiento de voz e IA quedan fuera del núcleo

Detalle de la incidencia del navegador

En este turno se levantó el servidor frontend de Vite en `http://127.0.0.1:5174/` y la API en `http://127.0.0.1:8001/`. Al delegar la aceptación al subagente de navegador con las instrucciones precisas para los roles ADMIN, PADRE y TERAPEUTA, la herramienta interna `open_browser_url` falló repetidamente:

text

failed to run playwright manager: failed to install playwright: could not install driver:

error: got non 200 status code: 404 (404 Not Found) from
`https://playwright.azureedge.net/builds/driver/playwright-1.57.0-win32_x64.zip`

Siguiendo las directrices del proyecto y las instrucciones expresas del usuario ("Si ocurre nuevamente, no lo eludas cambiando puerto, origen o herramienta"), no se realizaron evasiones ni se simuló falsamente una interacción visual en el navegador. Las 232 pruebas unitarias de Vitest validan el montaje, renderizado, validación de formularios y manejo de errores de los componentes en jsdom, pero la interacción E2E en navegador real permanece formalmente como pendiente.

06. Seguridad, incidentes y límites

Análisis del Incidente 01 histórico

En la auditoría inicial AUDIT-2026-10-08 / 2026-10-09-01, una ejecución no aislada de pruebas heredadas basadas en unittest (test_admin_crud.py, test_pacientes_crud.py) apuntó al puerto predeterminado 8000 (el cual conectaba a Supabase compartido), ejecutando creaciones y eliminaciones sobre cuentas de prueba (identificadas en los logs con IDs 44 y 45).

Estado y mitigaciones verificadas:

1. Las suites heredadas fueron inmediatamente deshabilitadas mediante unittest.skip condicionado a variables de entorno explícitas. En esta corrida se mantuvieron 25 pruebas omitidas.
2. Se implementaron guardas de tres capas:
 - testing_database.py: restringe las URLs de base de datos a 127.0.0.1 o localhost y exige prefijo ashakids_test_*
 - live_environment.py: restringe la API de pruebas a http://127.0.0.1:8001/api/v1 y valida coincidencia estricta de variables de entorno.
 - verify_delivery_journey.py (confirm_api_database): verifica directamente en la base de datos de PostgreSQL que el hash de sesión emitido por la API pertenezca a la instancia local antes de emitir la primera escritura clínica.
3. Impacto histórico en la base compartida:
 - Dado que los registros históricos carecían de marcas de tiempo por petición y no existía un snapshot de línea base anterior al incidente, el impacto residual en las cuentas de prueba de Supabase no ha sido modificado ni recalculado en este turno.
 - Se mantiene la recomendación al equipo de auditar directamente la tabla de usuarios en Supabase antes de la entrega final si esas cuentas de prueba se pretendían conservar.

07. Evidencia, pruebas y rúbrica

Comando / Verificación	Resultado exacto	Evidencia registrada
pytest completo (regress08)	119 aprobadas, 25 omitidas, 19 avisos (37.00s)	backend-tests.xml, backend-tests.txt
test_delivery_guard.py	4 aprobadas (0.23s)	Pruebas de guardia de sesión
Vitest componentes	232 aprobadas en 12 suites (29.76s)	frontend-tests.xml
Vitest enrutamiento	25 aprobadas en 5 suites	routing-tests.txt
Chequeo de tipos TypeScript	tsc --noEmit exitoso, 0 errores	typecheck.txt
Reglas de código frontend	318 archivos analizados, máx. 495 líneas	frontend-check.txt
Compilación Vite producción	Bundle 292.83 kB (gzip 92.55 kB)	build.txt
Mapa Graphify AST	1880 nodos AST, 102 comunidades, 0 secretos	knowledge-check.txt
Recorrido entrega API	92 respuestas HTTP conformes	http-delivery-journey.json
Muestra de reporte PDF	1 página, texto estructurado íntegro	reporte-sesion-sintetica.pdf, pdf-qa.json
Modelo físico inspeccionado	26 tablas, 178 columnas, 221 restricciones	schema-local.json

Todas las evidencias se encuentran archivadas y verificadas con SHA-256 en [docs/evidence/audit-2026-10-09-08/manifest.json](#).

08. Cierre y continuidad

La verificación del núcleo, PDF y mensajería en el entorno independiente de HailQueso demuestra la portabilidad del backend y la robustez de las APIs desacopladas de configuraciones particulares de una máquina.

Acciones inmediatas y siguientes pasos:

1. Mantener la fase 5 abierta respecto a la interacción en navegador real hasta resolver el gestor de Playwright en el entorno o realizar la validación visual interactiva en la sesión de usuario.
2. Publicar el avance revisado en la rama dev mediante el commit V14_Aceptacion_Nucleo_Reportes_Mensajes, conservando la secuencia de versiones sin reescribir historial.
3. Siguiendo tarea concreta propuesta: Preparación del paquete de entrega académica y consolidación de la documentación de rúbrica para las 18:00 Lima.